

A dark blue vertical bar on the left side of the page. A teal arrow points to the right from this bar, containing the date.

9/11/2026

TEKRA Studio

Технічне завдання: життєвий
цикл, тестування, Production та
управління Robot'ом

Several thin, dark blue wavy lines that originate from the vertical bar and curve upwards and to the right.

Oleksii Kulinskyi
TEKRA

Зміст

1. Мета	3
2. Загальна архітектура	4
2.1. Studio	5
2.2. Orchestrator	5
2.3. Assistant	6
3. Studio	7
3.1. Загальна модель	7
3.2. Рівні Studio	8
3.3. Custom components	8
4. Robot як основна сутність	10
4.1. Основні атрибути Robot'a	10
4.1.1. Robot ID	10
4.1.2. Metadata Robot'a	11
4.1.3. Version	12
4.1.4. Versioning rules	13
4.1.5. Структура Version[-s]	13
4.1.6. Lifecycle Robot'a	14
4.1.7. Authorization	16
4.1.8. Availability	17
4.1.9. Health	18
4.1.10. Active Production Version	19
4.1.11. Rollback	20
4.1.12. Test Period	20
4.1.13. TEST execution	21
4.1.14. TEST Period expiry	22
4.1.15. TEST → PROD	23
4.1.16. Execution	24
4.1.17. Структура Execution	25
4.1.18. Execution Status	25
4.2. Повна модель Robot → Version → Execution	26

4.3.	Принцип розділення станів.....	28
4.4.	Visibility.....	29
4.5.	Permissions.....	29
4.6.	TEST → PROD interactions.....	31
4.7.	Orchestrator.....	32
4.7.1.	Функції Orchestrator.....	32
4.7.2.	Pre-run checks.....	33
4.7.3.	Priority.....	34
4.7.4.	Admin Panel.....	35
4.7.5.	Audit.....	36
4.8.	Assistant.....	39
5.	Приклад lifecycle.....	40
6.	Приклад TEST.....	43
7.	Data model.....	45
8.	Responsibility matrix.....	48
9.	Acceptance Criteria.....	49
10.	Невирішені design decisions.....	51
11.	Фінальна концептуальна модель.....	53

1. Мета

Документ визначає цільову функціональну та логічну модель роботи Robot'a, його версій, тестування, переведення у PROD та виконання.

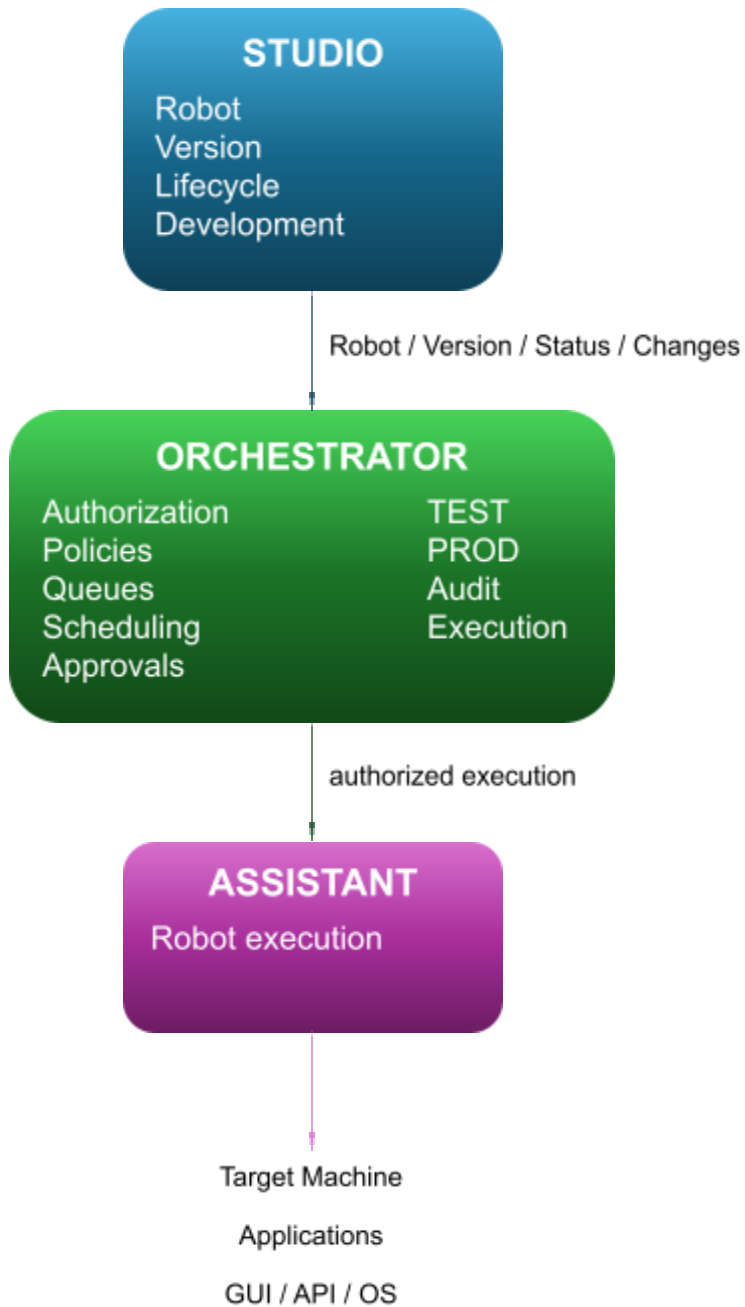
Система повинна забезпечувати:

- створення та редагування Robot'a у Studio;
- розробку та тестування Robot'a;
- багаторазове тестове виконання без необхідності погодження кожного запуску;
- створення та зберігання версій Robot'a;
- контрольований перехід конкретної Version у PROD;
- централізований контроль виконання через Orchestrator;
- виконання Robot через Assistant на цільовій машині;
- розділення lifecycle, authorization, availability та health;
- однозначне визначення конкретної Version, яка виконується;
- централізоване управління TEST та PROD;
- розмежування видимості та прав доступу;
- пріоритезацію PROD-виконань;
- повний аудит критичних дій.

Ключовий принцип: Lifecycle, Authorization, Availability, Health, Version та Execution Status - це різні характеристики Robot'a і не повинні об'єднуватися в один Status.

2. Загальна архітектура

Система складається з трьох основних компонентів:



2.1. Studio

Studio - web-застосунок, доступний через браузер. Studio використовується для:

- створення Robot'a;
- редагування Robot'a;
- створення Version[-s];
- роботи з компонентами;
- створення BPMN-процесів;
- тестування (через Orchestrator);
- ініціації переведення Robot/Version у PROD;
- перегляду доступної історії та станів відповідно до прав користувача.

2.2. Orchestrator

Orchestrator є центральним control layer системи. Він відповідає за:

- authorization;
- policy enforcement;
- TEST/PROD;
- approval;
- Test Period;
- queues;
- scheduling;
- execution control;
- пріоритети;
- availability;
- audit;
- передачу команд Assistant.

2.3. Assistant

Assistant - desktop application/agent, встановлений на машині, де фактично виконується/працює Robot. Assistant:

- отримує Execution від Orchestrator;
- виконує (start/stop/pause тощо) Robot'а;
- працює з GUI/API/OS залежно від компонентів Robot'а;
- передає технічний результат виконання назад в Orchestrator.

Assistant не повинен самостійно обходити policy та authorization Orchestrator'а.

3. Studio

3.1. Загальна модель

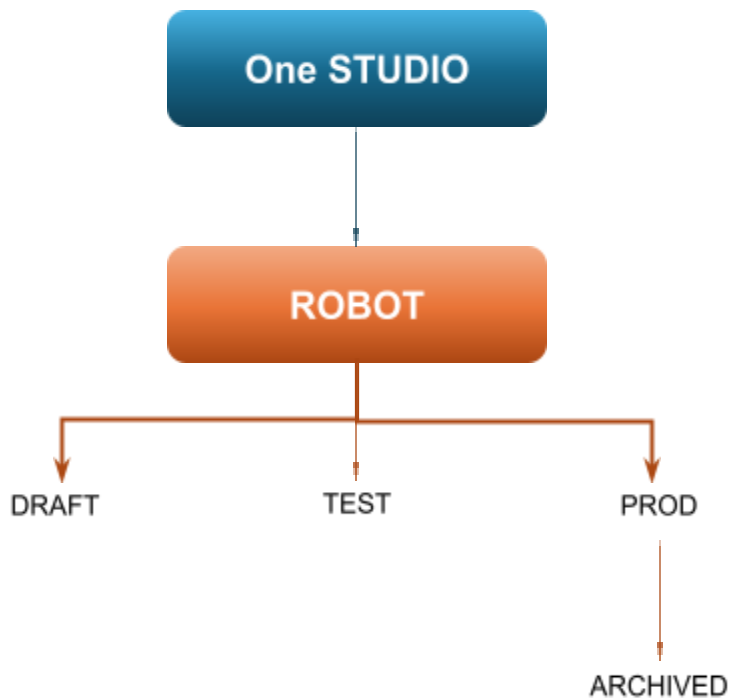
Існує **тільки одна Studio**.

Не створюються:

- TEST Studio;
- PROD Studio;
- окремі Studio для різних lifecycle.

TEST та PROD є логічними контекстами життєвого циклу та виконання, які контролюються Orchestrator'ом.

Studio працює з одним і тим самим Robot'ом протягом усього його життєвого циклу.



3.2. Рівні Studio

Level 1: BPMN

BPMN використовується для опису бізнес-процесу. В загальному виигляді BPMN складається з Task[-s]. Task - це **бізнес-задача**, яку виконує:

- людина;
- Robot;
- інша роль.

Task не є окремою низькорівневою дією Robot'а.

Level 2: Robot

Robot створюється з компонентів. Компоненти можуть працювати на рівнях:

- UI;
- API;
- Graph;
- OS;
- інше.

Level 3: Code

Embedding arbitrary code наразі не входить до поточного scope.

3.3. Custom components

Користувач повинен мати можливість створювати власні складені компоненти на базі базових компонентів. Наприклад, замість готового вузького:

Login to BAS

повинна бути можливість створити універсальну логіку:

Login to [selected system]

або складений користувацький компонент.

4. Robot як основна сутність

Robot - основна сутність, що представляє конкретну автоматизацію.

Robot має стабільну identity та власну історію Versions.

4.1. Основні атрибути Robot'a

Атрибут	Опис
Robot ID	Незмінний унікальний ідентифікатор
Name	Назва Robot'a
Description	Опис призначення Robot'a
Owner	Відповідальна особа/роль
Created By	Користувач, який створив Robot'a
Created At	Дата та час створення
Updated By	Користувач, який востаннє змінив Robot'a
Updated At	Дата та час останньої зміни
Lifecycle	DRAFT / TEST / PROD / ARCHIVED
Authorization	Поточний стан права на виконання
Availability	ENABLED / DISABLED / SUSPENDED
Health	HEALTHY / WARNING / ERROR / OFFLINE
Active Production Version ID	Посилання на конкретну активну PROD Version
Version[-s]	Набір усіх Version цього Robot'a

4.1.1. Robot ID

Robot ID генерується один раз під час створення Robot'a.

Наприклад: *RBT-000184*

Robot ID:

- не змінюється при перейменуванні Robot'a;
- не змінюється при створенні нової Version;
- використовується для однозначної ідентифікації Robot'a;
- використовується в Audit та Execution.

Наприклад:

Robot ID: *RBT-000184*

Name: *Обробка рахунків*

Version: *v27*

Зміна Name не створює нового Robot'a.

4.1.2. Metadata Robot'a

Robot повинен зберігати інформацію про власний життєвий цикл та походження.

Мінімально:

Robot ID
Name
Description
Owner

Created By
Created At

Updated By
Updated At

Lifecycle

Authorization

Availability

Health

Active Production Version ID

Важливо

Необхідно розділяти:

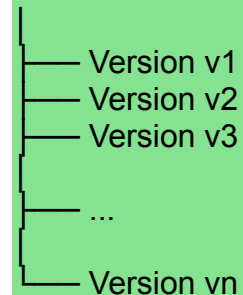
- хто створив **Robot'a**;
- хто створив **Version**;
- хто виконав **release**;
- хто виконав **approval**;
- хто запросив **Execution**.

Це різні події та різні аудиторські дані.

4.1.3. Version

Version - окрема сутність, що представляє **конкретну незмінну реалізацію Robot'a**. Robot може мати необмежену кількість Versions:

Robot *RBT-000184*



Version після створення є незмінною. Якщо потрібно змінити Robot'a - створюється нова Version.

4.1.4. Versioning rules

Нова Version створюється при збереженні змін, які можуть впливати на виконання Robot'a. До таких змін належать:

- логіка;
- code;
- components;
- sequence;
- conditions;
- parameters;
- variables;
- triggers;
- системи;
- ресурси;
- connection/configuration, якщо вони впливають на execution;
- інші execution-affecting settings.

Metadata-only changes

Окремо повинно бути визначено, які зміни вважаються metadata-only і не створюють нову Version. Це залишається design decision!!!

4.1.5. Структура Version[-s]

Мінімальна структура:

Поле	Опис
Version ID	Унікальний ID Version
Robot ID	Robot, до якого належить Version
Version Number	v1, v2, ... vn
Created By	Хто створив Version
Created At	Дата та час створення
Source Version ID	Version, на базі якої створена поточна
Change Description	Опис змін
Release State	Стан release
Authorization	Authorization саме цієї Version
Definition / Configuration	Повний snapshot реалізації
Definition Hash	Контроль цілісності
Released By	Хто виконав release
Released At	Дата та час release

Release State

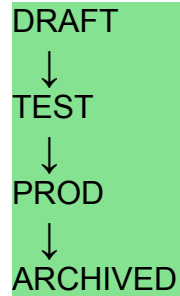
Пропоную використати наступні значення:

- DRAFT
- TESTING
- RELEASED
- SUPERSEDED
- REJECTED

Release State Version та Lifecycle Robot - **не одне й те саме. Це різні сутності.**

4.1.6. Lifecycle Robot'a

Lifecycle визначає етап життєвого циклу Robot'a.



DRAFT

Robot створений або налаштовується.

TEST

Robot знаходиться у тестуванні. Developer може:

- редагувати Robot'a;
- створювати Versions;
- запускати Robot;'а
- запускати окремі частини;
- повторювати executions;
- аналізувати результати.

PROD

Robot переведений у production lifecycle. **PROD не означає автоматично APPROVED.**

Можливий стан:

Lifecycle = PROD

Authorization = PENDING_APPROVAL

У такому випадку execution заборонений.

ARCHIVED

Robot більше не використовується. Він зберігається для:

- історії;
- audit;
- аналізу;
- відновлення інформації.

Execution ARCHIVED Robot заборонений.

4.1.7. Authorization

Authorization - окремий вимір стану Robot'a/Version.

Значення:

Status	Значення
NOT_REQUIRED	Погодження не потрібне
PENDING_APPROVAL	Очікується погодження
APPROVED	Погодження надано
REJECTED	Погодження відхилено
REVOKED	Раніше наданий дозвіл відкликано
EXPIRED	Термін дії дозволу завершився

Ключове правило

Production authorization прив'язується до **конкретної Version**. Наприклад:

RBT-000184

Version: v27

Lifecycle: PROD

Authorization: APPROVED

Якщо створена v28:

v27 → APPROVED

v28 → NOT APPROVED

Approval v27 не переноситься автоматично на v28.

4.1.8. Availability

Availability відповідає на питання: чи дозволено Robot'а виконувати зараз?

Значення:

- ENABLED
- DISABLED
- SUSPENDED

ENABLED

Robot може виконуватися, якщо всі інші перевірки пройдені.

DISABLED

Robot навмисно вимкнений.

SUSPENDED

Robot тимчасово призупинений через:

- інцидент;
- системне правило;
- security issue;
- іншу визначену policy.

Availability незалежна від Authorization.

Наприклад:

Lifecycle: PROD

Authorization: APPROVED

Availability: DISABLED

є валідним станом.

4.1.9. Health

Health описує технічний стан. Значення:

- HEALTHY
- WARNING
- ERROR
- OFFLINE

Health не повинен автоматично змінювати lifecycle.

Наприклад:

Lifecycle: PROD

Authorization: APPROVED

Availability: ENABLED

Health: ERROR

є валідною комбінацією.

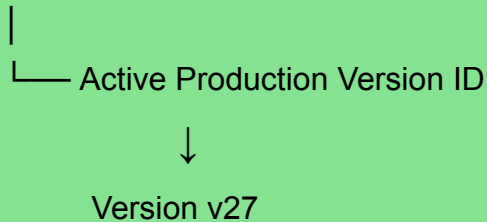
4.1.10. Active Production Version

Для кожного Robot'а повинна бути однозначно визначена:

Active Production Version

Це **не окремий статус**. Це pointer:

Robot



Наприклад:

Robot ID: *RBT-000184*

Active Production Version ID: *VER-000821*

Version Number: *v27*

При цьому Robot може одночасно мати:

v27 → Active PROD

v28 → TEST / development

Створення v28 не повинно автоматично замінювати v27.

4.1.11. Rollback

Rollback - повернення Active Production Version до попередньої стабільної Version.

Наприклад:

До rollback:

Active PROD → v27

Після rollback:

Active PROD → v26

Rollback:

- не створює нову Version;
- не змінює Definition v26;
- змінює Active Production Version pointer;
- повинен бути зафіксований в Audit.

Точний механізм approval rollback залишається окремим design decision.

4.1.12. Test Period

Test Period - окремий дозвіл на виконання Robot'a у TEST протягом визначеного часу. Він потрібен **лише якщо TEST Approval Required = ON**.

Основні параметри

Параметр	Опис
----------	------

ID	ID Test Period
Robot ID	Robot
Approved By	Admin
valid_from	Початок
valid_to	Завершення
Status	Статус періоду
Revoke	Можливість дострокового відкликання

Можливі тривалості:

- 1h;
- 3h;
- 1d;
- 2d;
- 1 week;
- custom.

Набір значень конфігурується Admin'ом.

4.1.13. TEST execution

Robot у TEST повинен дозволяти Developer'у виконувати багаторазові executions.

Наприклад:

TEST Period = 1 day

Version v10

Version v11

Version v12

Execution #1

Execution #2

...

Execution #800

Новий approval перед кожним execution **не потрібен**. Під час активного Test Period Developer може:

- змінювати Robot'а;
- створювати нові Versions;
- тестувати окремі частини;
- тестувати повного Robot'а;
- виконувати багато запусків.

4.1.14. TEST Period expiry

Після завершення Test Period:

New TEST execution → BLOCKED

Для вже запущених Execution поведінка визначається Admin policy:

ALLOW_TO_FINISH: вже запущений Execution завершується сам, не примусово.

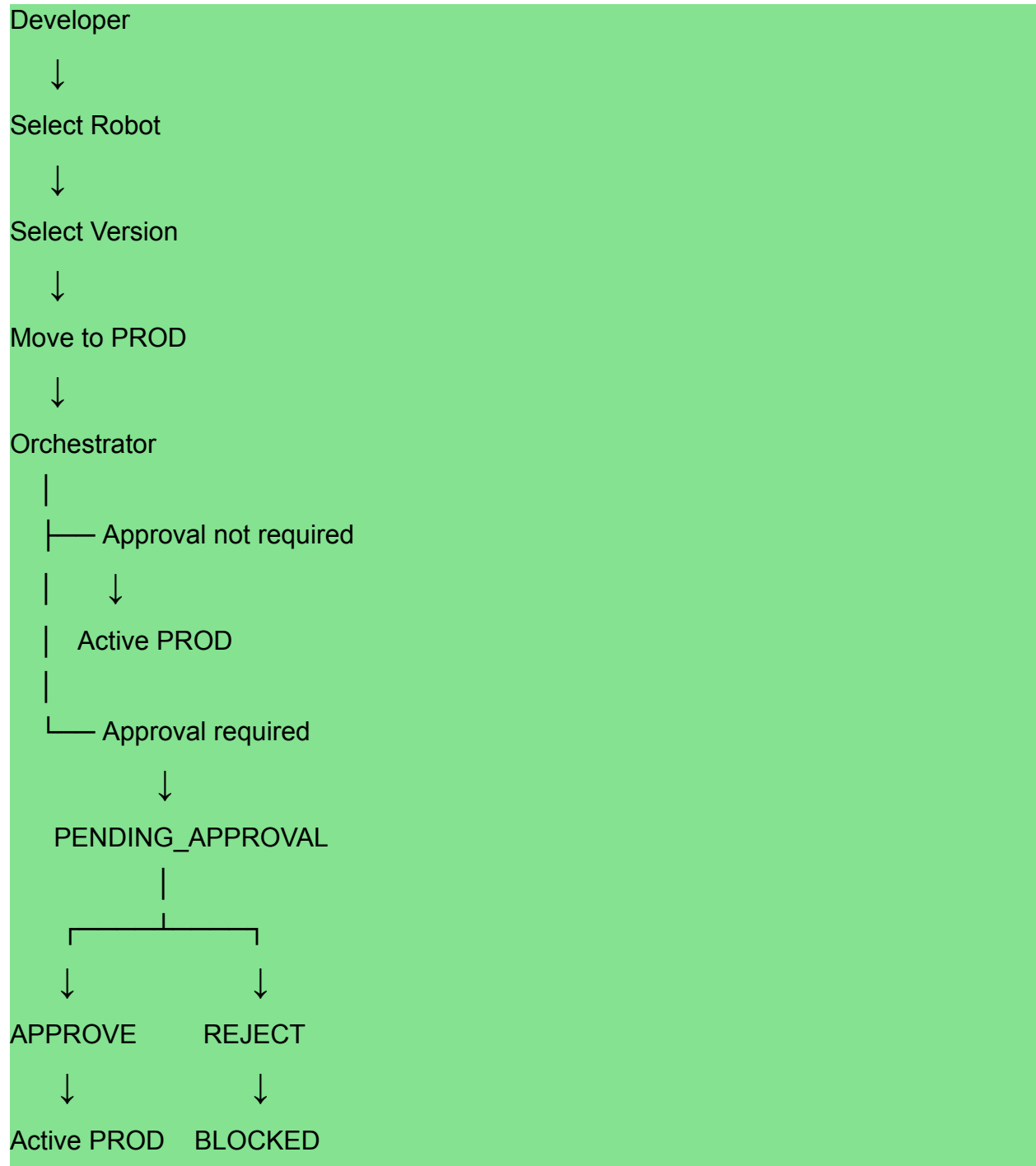
Новий - не починається.

TERMINATE: вже запущений Execution завершується примусово. Нові - не починаються.

Admin може відкликати активний Test Period достроково.

4.1.15. TEST → PROD

Developer ініціює переведення Robot'a у PROD.



Алгоритм дій

1. Developer обирає Robot'а
2. Developer обирає конкретну Version
3. Developer натискає Move to PROD
4. Studio передає до Orchestrator:
 - 4.1. Robot ID
 - 4.2. Version ID
 - 4.3. користувача
 - 4.4. час
 - 4.5. параметри request
5. Orchestrator перевіряє PROD policy
6. Якщо approval не потрібен - Version може стати Active Production Version
7. Якщо approval потрібен - Authorization = PENDING_APPROVAL
8. Execution заблокований до рішення
9. Admin виконує APPROVE або REJECT
10. APPROVE → Version може стати Active Production Version
11. REJECT → execution заблокований
12. Всі дії потрапляють в Audit

4.1.16. Execution

Execution - **окрема сутність**, яка представляє конкретний запуск конкретної Version. Це принципово важливо - Robot не повинен мати статус:

RUNNING

як характеристику самого Robot'а.

RUNNING - статус конкретного Execution.

Один Robot може мати:

Execution #1001 → v26 → RUNNING

Execution #1002 → v26 → COMPLETED

Execution #1003 → v27 → QUEUED

Execution #1004 → v25 → FAILED

одночасно.

4.1.17. Структура Execution

Поле	Опис
Execution ID	Унікальний ID запуску
Robot ID	Robot
Version ID	Конкретна Version
Lifecycle	TEST / PROD
Requested By	Хто запросив запуск
Requested At	Коли запросив
Assistant	Який Assistant (десктопний застосунок)
Host	На якій машині
Started At	Початок виконання
Finished At	Завершення
Execution Status	Статус запуску
Result	Результат
Error	Помилка, якщо є

Ключове правило: кожен Execution повинен однозначно посилатися на Robot ID + Version ID.

Це дозволяє визначити, **яка саме реалізація Robot'а фактично виконувалась.**

4.1.18. Execution Status

Status	Значення
QUEUED	Execution поставлено в чергу
RUNNING	Виконується
PAUSED	Призупинено
COMPLETED	Успішно завершено
FAILED	Завершено з помилкою
CANCELLED	Скасовано

Базовий flow:



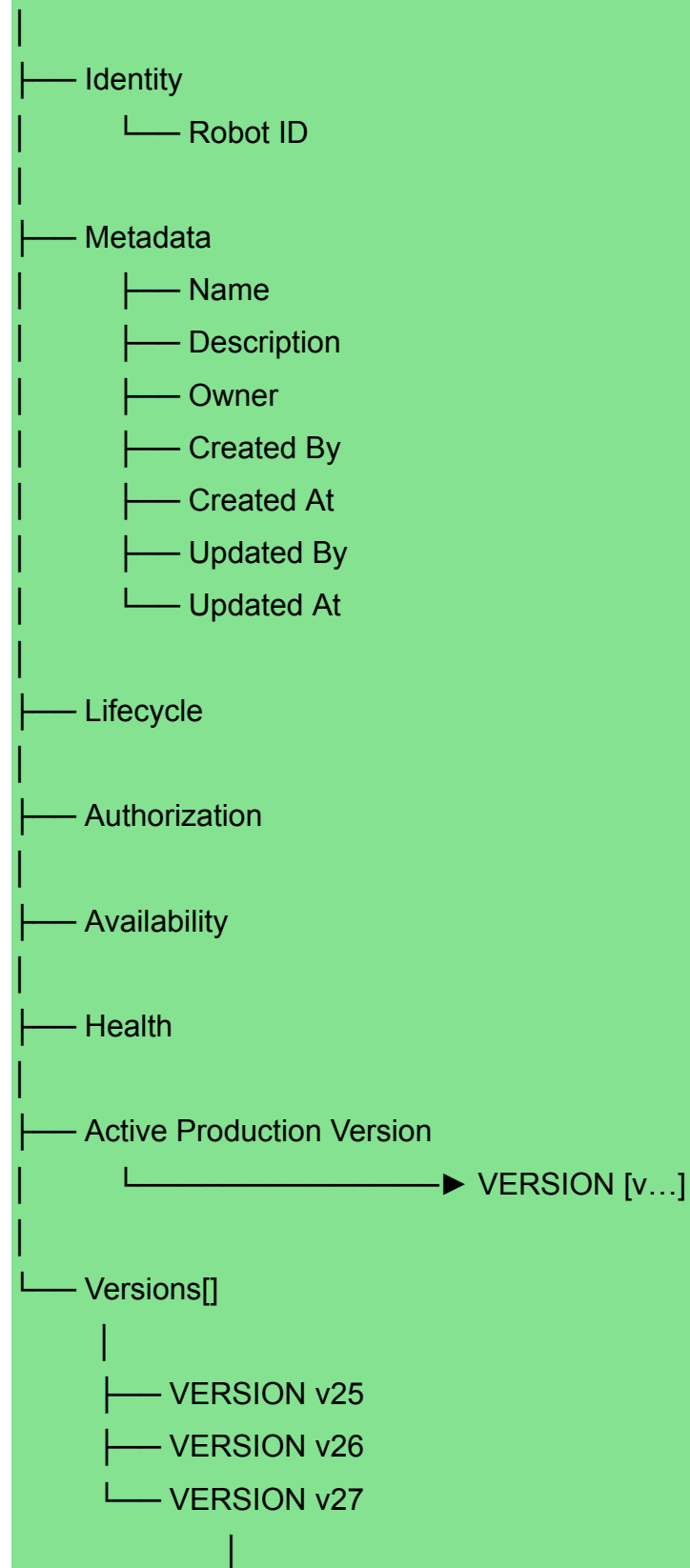
PAUSED може бути проміжним станом. Окремо потрібно визначити поведінку

RUNNING Execution при:

- DISABLED;
- SUSPENDED;
- REVOKED;
- втраті Assistant;
- втраті connection.

4.2. Повна модель Robot → Version → Execution

ROBOT



- └─ Definition
- └─ Release State
- └─ Authorization
- └─ ...

EXECUTION[]

- |
- └─ Execution #1001 → Robot + Version v27
- └─ Execution #1002 → Robot + Version v27
- └─ Execution #1003 → Robot + Version v26

4.3. Принцип розділення станів

Сутність / атрибут	Питання
Robot	Що це за автоматизація?
Robot ID	Який саме Robot?
Version	Яка конкретна реалізація Robot'a?
Active Production Version	Яка Version зараз активна у PROD?
Lifecycle	На якому етапі життєвого циклу Robot?
Release State	На якому етапі release знаходиться Version?
Authorization	Чи дозволено виконання?
Availability	Чи дозволено виконання зараз?
Health	Який технічний стан?
Execution	Який конкретний запуск?
Execution Status	Що зараз відбувається з цим запуском?

Не допускається зводити всі ці поняття в один статус!!!

4.4. Visibility

Visibility та Permission - різні поняття.

Користувач може:

- бачити Robot'a;
- але не мати права його редагувати;
- бачити Version;
- але не мати права її запускати;
- бачити PROD Robot'a;
- але не мати права виконання.

Базова модель Visibility:

Object	Regular User	Developer	Admin
DRAFT Robot	Hi	Так	Так
TEST Robot	Hi	Так	Так
PROD Robot	Так, за RBAC	Так	Так
ARCHIVED Robot	Hi / policy	Так	Так
TEST Version	Hi	Так	Так
PROD Version	Так, за RBAC	Так	Так
TEST Execution	Hi / власні	Так	Так
PROD Execution	Так / дозволені	Так / дозволені	Так
Approval	Статус	Статус	Так
Test Period	Hi	Власний/requested	Так
Audit	Hi	Обмежено	Так

4.5. Permissions

Базовий перелік permissions:

- **Robot**

- ROBOT_VIEW
- ROBOT_CREATE
- ROBOT_EDIT
- ROBOT_DELETE
- ROBOT_TEST
- ROBOT_MOVE_TO_PROD
- ROBOT_ARCHIVE

- **Version**

- VERSION_VIEW
- VERSION_CREATE
- VERSION_EDIT

Фактична Version є незмінною після створення; VERSION_EDIT може використовуватися для draft/version metadata до моменту фіксації.

- **Execution**

- EXECUTION_VIEW
- EXECUTION_RUN
- EXECUTION_CANCEL

- **Production**

- PROD_APPROVE
- PROD_REJECT
- PROD_REVOKE

- **TEST**

- TEST_PERIOD_REQUEST
- TEST_PERIOD_APPROVE
- TEST_PERIOD_REVOKE

- **Availability**

- ROBOT_ENABLE

- ROBOT_DISABLE
- ROBOT_SUSPEND
- ROBOT_RESUME
- **Administration**
 - POLICY_VIEW
 - POLICY_EDIT
 - AUDIT_VIEW

Фінальна RBAC-матриця є окремим design decision.

4.6. TEST → PROD interactions

TEST Robot не повинен напряму обходити Orchestrator для взаємодії з PROD Robot.

Базова policy:

TEST → PROD = DENY ALL

Можуть існувати exceptions.

Наприклад:

RBT-TEST-001 → RBT-PROD-015 = READ

RBT-TEST-002 → RBT-PROD-020 = CALL + READ

Доступні permissions:

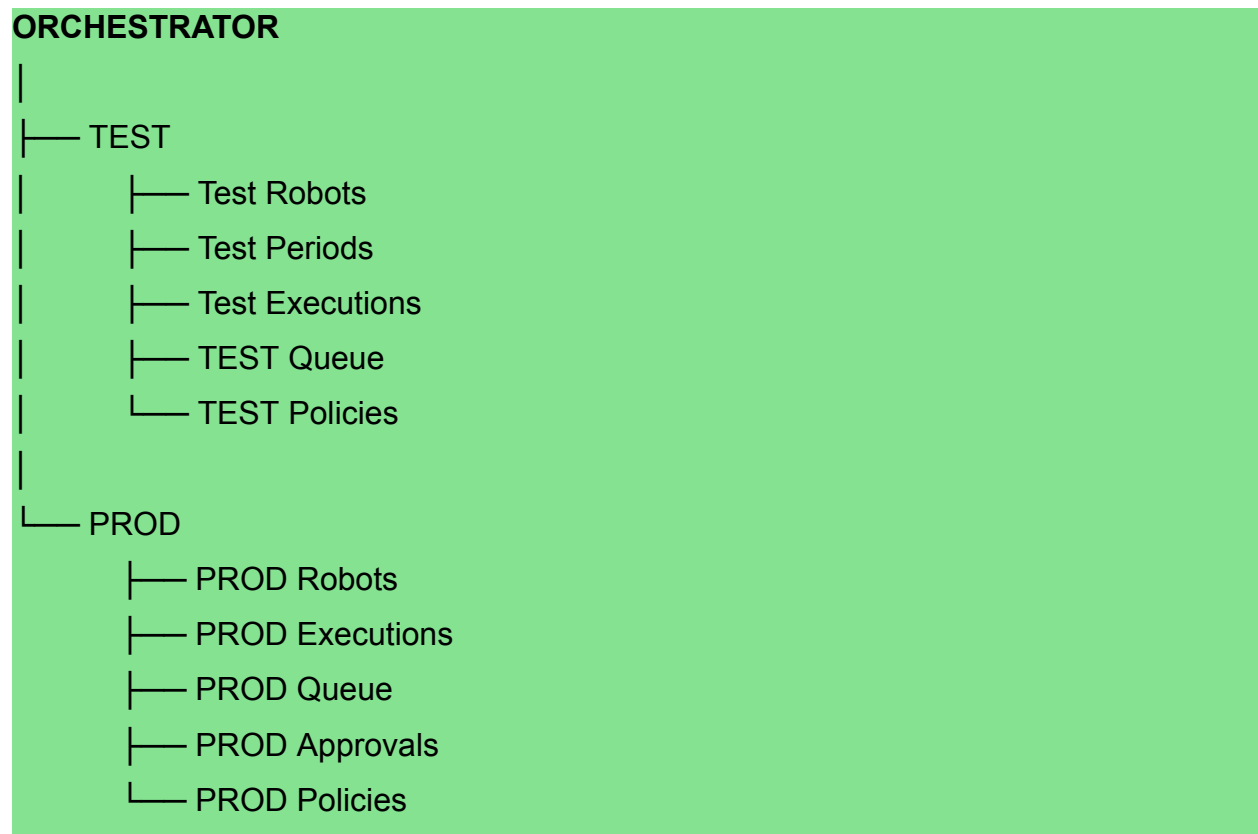
Permission	Значення
------------	----------

READ	Читання визначених даних/стану
CALL	Виклик визначеного PROD Robot'а
WRITE	Дозволений запис/зміна
TRIGGER	Запуск/тригер
ALL	Всі взаємодії дозволені
DENY ALL	Всі взаємодії заборонені

Точна семантика кожного permission повинна бути визначена окремо.

4.7. Orchestrator

Orchestrator - центральний компонент контролю:



Фізично це буде одна інсталяція.

4.7.1. Функції Orchestrator

Orchestrator повинен:

- приймати Robot/Version information від Studio;
- зберігати authorization;
- перевіряти policy;
- контролювати Test Period;
- контролювати PROD approval;
- контролювати Availability;
- контролювати TEST → PROD interactions;
- керувати queue;
- керувати scheduling;
- визначати priority;
- передавати Execution Assistant;
- блокувати несанкціоновані executions;
- вести Audit.

Ключовий принцип:

Admin decides → Orchestrator controls/enforces → Assistant / Robot executes.

4.7.2. Pre-run checks

Перед кожним Execution Orchestrator повинен перевірити:

1. Robot існує
2. Version існує
3. Version належить цьому Robot'у
4. Lifecycle дозволяє execution

5. Authorization дозволяє execution
6. Authorization не EXPIRED / REVOKED
7. Availability = ENABLED
8. Robot не SUSPENDED
9. Для TEST існує чинний Test Period, якщо policy цього вимагає
10. Assistant доступний
11. Host доступний
12. Необхідні ресурси доступні
13. TEST → PROD policy не порушена
14. Користувач має EXECUTION_RUN
15. Scheduling policy дозволяє запуск
16. Concurrency limits дозволяють запуск
17. Інші system/security policies дозволяють запуск

Якщо будь-яка критична перевірка не пройдена:

Execution → BLOCKED

і причина повинна бути зафіксована в Audit.

4.7.3. Priority

PROD Executions мають вищий пріоритет за TEST Executions. Наприклад:

100 TEST jobs

+

1 PROD job

Orchestrator повинен забезпечити обробку PROD відповідно до встановленої scheduling policy. При цьому повинні бути визначені:

- concurrency;

- resource limits;
- starvation prevention;
- queue policy;
- resource locking;
- max parallel executions;
- поведінка при перевантаженні Assistant.

4.7.4. Admin Panel

Admin Panel знаходиться в Orchestrator. Admin повинен мати можливість налаштовувати:

TEST

- TEST Approval Required ON/OFF;
- дозволені Test Period;
- максимальну тривалість Test Period;
- поведінку після expiry:
 - ALLOW_TO_FINISH;
 - TERMINATE.

PROD

- PROD Approval Required ON/OFF;
- approval;
- reject;
- revoke.

Availability

- Enable;
- Disable;

- Suspend;
- Resume.

TEST → PROD

- Global policy;
- exceptions для конкретних Robot.

Scheduling

- priority;
- concurrency;
- resource limits;
- queue policy.

Audit

- перегляд audit'у;
- фільтрація;
- пошук.

4.7.5. Audit

Orchestrator повинен вести Audit усіх критичних дій.

Robot

- створення;
- редагування;
- перейменування;
- зміна Owner'а;
- lifecycle transition;
- archive.

Version

- створення;
- release;
- promotion;
- production activation;
- supersede;
- rollback.

Authorization

- request;
- approve;
- reject;
- revoke;
- expire.

Test Period

- create;
- approve;
- revoke;
- expire.

Availability

- enable;
- disable;
- suspend;
- resume.

Execution

- request;

- block;
- queue;
- start;
- pause;
- complete;
- fail;
- cancel.

Policy

- створення;
- зміна;
- активація;
- деактивація.

Audit Event

Мінімально:

- Who
- When
- Action
- Robot ID
- Version ID
- Previous State
- New State
- Reason
- Execution ID
- Assistant
- Host
- Lifecycle

4.8. Assistant

Використовується **один Assistant** для TEST та PROD.

Assistant отримує Execution Context:

TEST

або

PROD

Assistant:

- отримує execution від Orchestrator'а;
- завантажує конкретну Version;
- виконує її;
- працює з цільовою машиною;
- повертає результат;
- передає технічні події виконання.

Assistant не повинен мати можливості обійти:

- authorization;
- availability;
- policy;
- approval;
- queue.

5. Приклад lifecycle

Розглянемо Robot'a:

Robot ID: *RBT-000184*
Name: *Обробка рахунків*

Крок 1

Developer створює Robot.

Lifecycle = DRAFT

Зберігається:

Created By
Created At

Крок 2

Developer переводить Robot'a у TEST.

Lifecycle = TEST

Крок 3

Створюється:

Version v10

Крок 4

Developer тестує Robot'a.

Створюються:

Execution #1001
Execution #1002
...
Execution #1100

Крок 5

Створюються:

v11
v12

Крок 6

Developer тестує v12.

v12
Authorization = TEST authorization

Крок 7

Developer вибирає:

RBT-000184
v12

і натискає:

Move to PROD

Крок 8

Orchestrator перевіряє PROD policy.

Якщо approval потрібен:

Lifecycle = PROD

Authorization = PENDING_APPROVAL

Крок 9

Admin погоджує:

Authorization = APPROVED

Крок 10

Active Production Version = v12

Крок 11

Regular User може бачити PROD Robot'a та запускати його відповідно до RBAC.

Крок 12

Developer починає роботу над наступною версією:

v13

При цьому:

Active PROD = v12

і approval v12 **не переноситься автоматично на v13.**

6. Приклад TEST

Robot:

```
RBT-000184  
Lifecycle = TEST
```

Policy:

```
TEST Approval Required = ON
```

Admin надає:

```
Test Period  
valid_from = 10:00  
valid_to = 18:00
```

Developer:

- змінює Robot'a;
- створює v10;
- створює v11;
- створює v12;
- запускає Robot'a 800 разів;
- тестує окремі частини;
- аналізує failures.

Новий approval перед кожним execution не потрібен.

О 18:00:

```
Test Period = EXPIRED
```

Нові executions:

BLOCKED

7. Data model

Robot

```
Robot {  
  RobotID  
  Name  
  Description  
  Owner  
  
  CreatedBy  
  CreatedAt  
  UpdatedBy  
  UpdatedAt  
  
  Lifecycle  
  
  Authorization  
  Availability  
  Health  
  
  ActiveProductionVersionID  
  
  Versions[]  
}
```

Version

```
Version {  
  VersionID  
  RobotID  
  Number
```

```
    CreatedBy
    CreatedAt

    SourceVersionID
    ChangeDescription

    ReleaseState
    Authorization

    Definition
    DefinitionHash

    ReleasedBy
    ReleasedAt
}
```

TestPeriod

```
TestPeriod {
    ID
    RobotID

    ApprovedBy

    ValidFrom
    ValidTo

    Status
    RevokedAt
}
```

Execution

```
Execution {  
  ExecutionID  
  
  RobotID  
  VersionID  
  
  Lifecycle  
  
  RequestedBy  
  RequestedAt  
  
  Assistant  
  Host  
  
  Status  
  
  StartedAt  
  FinishedAt  
  
  Result  
  Error  
}
```

8. Responsibility matrix

Дія	Developer	Orchestrator	Admin	Assistant
Створити Robot'a	Так	Зберігає / контролює	-	-
Редагувати Robot'a	Так	Контроль policy	-	-
Створити Version	Так	Зберігає	-	-
Тестувати	Ініціює	Дозволяє / блокує	Test Period, якщо потрібно	Виконує
Move to PROD	Ініціює	Обробляє	-	-
Approve PROD	-	Enforce	Так	-
Reject PROD	-	Enforce	Так	-
Production execution	Запитує за RBAC	Авторизує / Queue	Policy	Виконує
Disable Robot	Permission	Enforce	Permission	-
Suspend Robot	Permission	Enforce	Permission	-
Audit	-	Так	Перегляд	Передає runtime data

9. Acceptance Criteria

Система вважається такою, що відповідає ТЗ, якщо:

1. Robot має незмінний Robot ID
2. Robot має Name, Description, Owner
3. Robot має Created By / Created At
4. Robot має Updated By / Updated At
5. Version є окремою сутністю
6. Robot має набір Versions
7. Version має власний Version ID
8. Version має Created By / Created At
9. Version є незмінною після створення
10. Нова execution-affecting зміна створює нову Version
11. Active Production Version є pointer на конкретну Version
12. Active Production Version не є окремим lifecycle status
13. Lifecycle підтримує DRAFT / TEST / PROD / ARCHIVED
14. Authorization є окремим виміром
15. Availability є окремим виміром
16. Health є окремим виміром
17. PROD може мати PENDING_APPROVAL
18. PROD execution неможливий без необхідного approval
19. Approval прив'язаний до конкретної Version
20. Нова Version не успадковує автоматично PROD approval попередньої
21. TEST підтримує багаторазові executions
22. TEST не вимагає approval перед кожним execution
23. Test Period має valid_from / valid_to
24. Test Period може бути revoked
25. Після expiry нові TEST executions блокуються
26. Поведінка вже запущених executions після expiry задається policy
27. Один Assistant використовується для TEST та PROD
28. Orchestrator контролює execution

29. Assistant не обходить Orchestrator
30. TEST та PROD логічно розділені в Orchestrator
31. PROD має вищий priority за TEST
32. Execution є окремою сутністю
33. Execution посилається на конкретні Robot ID + Version ID
34. Regular User не бачить DRAFT/TEST за базовою моделлю
35. Developer бачить Robot'а відповідно до RBAC
36. Admin має повну visibility
37. TEST → PROD interaction контролюється policy
38. Критичні дії записуються в Audit
39. Rollback не створює нову Version
40. Rollback змінює Active Production Version

10. Невирішені design decisions

Наступні питання поки не повинні бути «вигадані» в ТЗ як уже затверджені:

1. Які саме зміни є metadata-only
2. Чи можуть одночасно існувати кілька APPROVED PROD Versions
3. Чи може одночасно виконуватися кілька Versions одного Robot'a
4. Точний approval flow для rollback
5. Хто має право ініціювати rollback
6. Чи потребує rollback окремого approval
7. Чи можна відновлювати ARCHIVED Robot'a
8. Точна семантика READ / CALL / WRITE / TRIGGER
9. Чи може TEST викликати PROD виключно через Orchestrator API
10. Умови автоматичного SUSPENDED
11. Поведінка RUNNING Execution після DISABLED
12. Поведінка RUNNING Execution після SUSPENDED
13. Поведінка RUNNING Execution після REVOKED
14. Фінальна RBAC-матриця
15. Scheduling algorithm
16. Concurrency model
17. Starvation prevention
18. Resource locking
19. Resource reservation
20. Deployment нової PROD Version без downtime
21. Точний UX lifecycle/version statuses у Studio
22. Поведінка working Version при одночасному Active Production Version
23. Точна модель Version Release State
24. Чи потрібен окремий DEPRECATED / RETIRED lifecycle
25. Чи має TEST Version окремий authorization object, чи тільки Test Period
26. Чи може Test Period охоплювати нові Versions, створені після його approval
27. Що відбувається при створенні нової Version під час активного PROD

28. Яким чином Orchestrator визначає конкретний runtime package/Definition для Assistant

11. Фінальна концептуальна модель

У системі необхідно чітко розділяти:

IDENTITY



Robot ID



IMPLEMENTATION



Version



LIFECYCLE

DRAFT / TEST / PROD / ARCHIVED



|— AUTHORIZATION

| NOT_REQUIRED / PENDING / APPROVED / ...



|— AVAILABILITY

| ENABLED / DISABLED / SUSPENDED



|— HEALTH

HEALTHY / WARNING / ERROR / OFFLINE

Robot

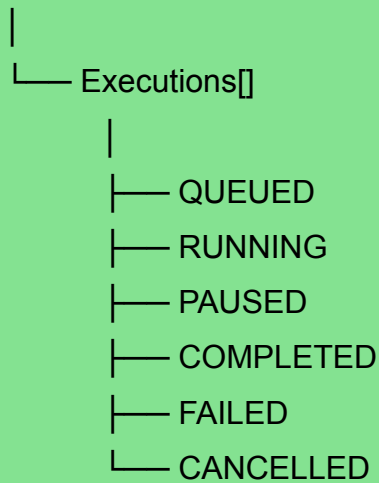


|— Active Production Version



Version

Version



Основний принцип:

Robot визначає, що це за автоматизація.

Version визначає, яка саме реалізація автоматизації.

Lifecycle визначає етап життєвого циклу Robot'а.

Authorization визначає, чи дозволене виконання.

Availability визначає, чи дозволене виконання зараз.

Health визначає технічний стан.

Active Production Version визначає, яка Version зараз працює як PROD.

Execution визначає конкретний запуск конкретної Version.

І вся операційна модель зводиться до:

Developer



Studio



Robot / Version / request



Orchestrator

|

|— Lifecycle check

|— Authorization check

|— Availability check

|— Test Period check

|— Policy check

|— Permission check

|— Resource check

|— Queue

|— Scheduling

↓

Assistant

↓

Target Machine